# 1 The Design

A design is a closed semantic world. It contains participants whose state is transformed through the design's unambiguous behaviors. When the environment interacts with the design, it introduces a change to the state of the world. This state transition may enable new behaviors, which the design deterministically resolves in response to the updated state, producing further state changes. When resolving a state, the design may either exhibit behavior directly or deterministically select which behaviors are enabled as a reaction to the state. The design continues reacting to each resulting state transition until no further behavior is enabled and the world reaches a stable semantic state. This chain of resolutions constitutes the structure of the design itself and provides the environment a fully specified path through it.

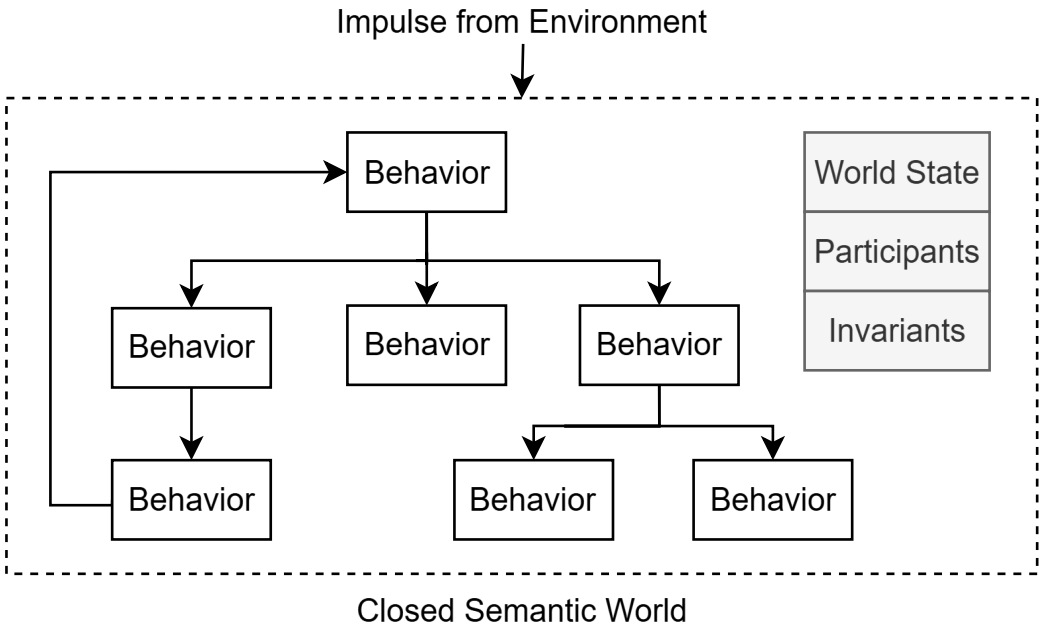


Figure 1: The Design

## 1.1 Semantic Invalidity

The design is said to be in a semantically invalid state when one of its intentions can't be realized as a result of that state. When the environment provides an input into this closed semantic world, in order to continue successfully realizing its intentions, the design must prevent semantically invalid states from persisting in its world.

In order for the closed semantic world to participate in reality, it must be implemented on a substrate. When realized on a substrate, reality can refute the assumptions of the closed semantic world and the design fails because its intentions can't be realized. As a result, the design must learn new semantics to continue realizing its intentions. So in a closed semantic world, a failure can be understood as an assumption made by the design that reality refutes and I will be using the world failure in this context for the rest of the document. When the implementation on a substrate halts because semantic invalidity was allowed to persist, I will refer to this as a mechanical failure.

To prevent semantically invalid states from persisting in its reality, semantic invariants act as markers that identify the semantically invalid state and predict the failure of downstream behaviors. This establishes an invariant path from the semantically invalid state to the intention that can't be realized and in order to restore semantic validity, the design must provide a semantically valid path to resolve the state.

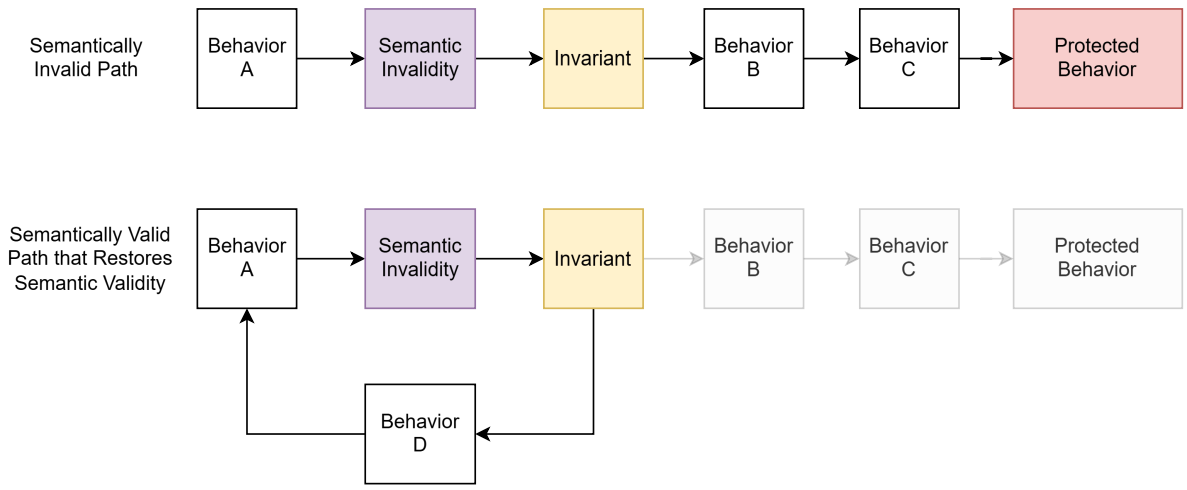


Figure 2: Invariant Path

## 1.2 Narrative of the World

Another way of describing a closed semantic world is through the narratives that are established by the design. The narratives are constructed using the behaviors and how it unambiguously modifies the state of the worlds participants. When looked at in this way, a semantically invalid path is a narrative that will not succeed in reality. When the design learns new semantics, it absorbs the awareness of the invalid narrative into its domain knowledge. Another way to say this is that the there exists a narrative in the world that is aware of the invalid narrative and it prevents any impulse from the environment from reaching it.

## 1.3 Reasoning about Closed Semantic World

Since the design is a closed semantic world, it is possible to reason about the world. For example, the invariants placed on a behavior can be used to reason about where to place the semantic invariants. This is possible because the narrative of the world unambiguously identifies where the participants enter a semantically invalid state and will inevitably reach the behavior. Furthermore, it is possible to identify the path that should be established to restore the semantically valid state.

This form of reasoning can be used to perform many meaningful actions on the world and I will list a few below:

- The ability to place invariants and resolve the semantically invalid states is failure prediction and resolution within the closed semantic world.
- The ability to walk down semantically invalid narrative paths and identify if the design selects a semantically valid narrative is testing the world.
- If all the invariants are tested but the design still fails, the failure is automatically diagnosed because it can only mean that new semantics must be learnt.

Together, these abilities establish debugging as a process of reasoning about the world given that the designs meaning is faithfully represented by its implementation. The kinds of reasoning that are possible are only limited by the meaning established by the design.

If the closed semantic world was established in an unambiguous and structured form, then an engine would be able to automatically reason about the world. In the next section, I will discuss the implications of this capability.

## 2 Semantic Reasoning Engine

As the previous section established, the design is a structure that can be reasoned about through the meaning that it establishes. As such, if the design is established in a structured and unambiguous form, an engine can automatically perform the semantic reasoning.

To make this possible, the design must be written in a Design Abstraction Language (DAL). The DAL is a declarative language that is used fully establish the closed semantic world. The scope of the DAL includes all the meaning needed to fully define the world (behvaiors, participants, attributes, invariants, etc).

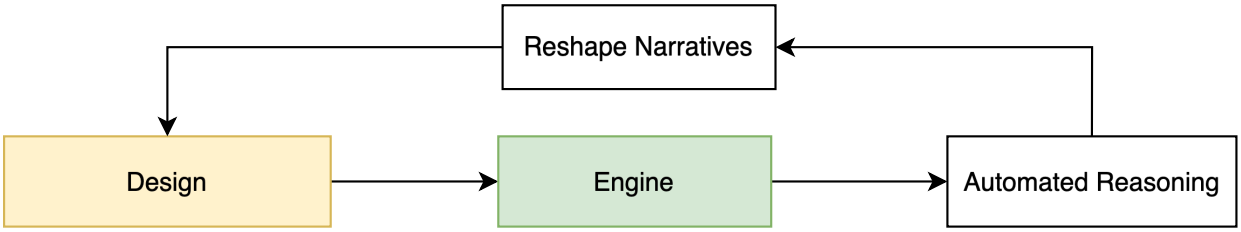


Figure 3: Automated Reasoning to Reshape Narrative

An engine can use the DAL definition to automatically reason about the world. It can use the meaning of the design to enforce its own definition of correctness. The ability of the engine to reason about the world is limited by the meaning that has been established. Any meaningful qquesiton about the world that can't be answered results in semantic expansion of the world.

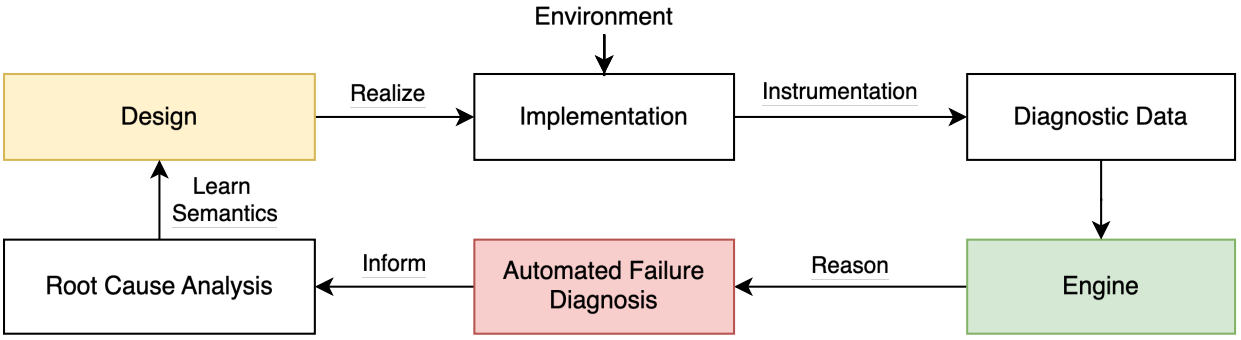


Figure 4: Ideal learning loop by reasoning about execution using design meaning

One of the powerful implications of this is that since the engine reasons about the world, it can eliminate any semantically invalid narratives from the world. This means that using the execution of the design, the engine can reason to automatically diagnosed any failures. A failure can only mean that the design must learn new semantics. This is possible because that the same engine can use the meaning of the constructed world to automatically diagnose the failure in the execution and identify where the semantics of the world has to expand through root cause analysis.

To do this, the following steps are neessary:

- The implementation and its execution must represent the design semantics faithfully.
- The execution must be losslessly preserved for the reasoning to be deterministic.

If the semantic gap between the implementation and the design is eliminated and the execution is losslessly preserved, it would enable a determinstic learning loop. More importantly, as a result of this, any meaningful question about the world can be automatically answered or an automatic process would collect the necessary diagnostic information needed to learn how to answer the question.

In the following sections, I will explain how a Computable Semantic Model (CSM) can be used to eliminate the semantic gap between the design and the implementation. Then I will explain how domain specific compression can be used to losslessly preserve all the narratives experienced in the world and finally I will explain how this fully automates systems management.

# 3 Computable Semantic Model

In order for the engine to reason about the execution of the design, the semantic gap between the design and the implementation must be eliminated. If this is not done, then the engine cannot deterministically reason about the world because there will be no way to know if the semantics need to expand or if the implementation doesn't realize themeaning of the design.

If the implementation does realize the meaning of the design faithfully, then all the automation that is enabled by the engine reasoning about the design can be used by the engine. To make this possible, the design must be established as a Computable Semantic Model (CSM) that is built from Computable Semantic Primitives (CSP).

A CSM is built by establishing the realization of the designs meaning. By specifying the design in this way, the design itself becomes executable. This means that the execution of the design is